



University
of Basel

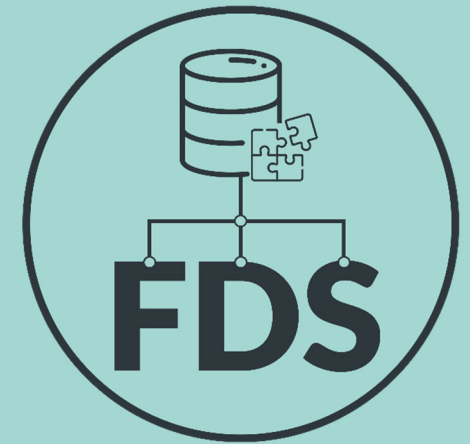
Chapter 4: Data

4.3: Replication

Foundations of Distributed Systems

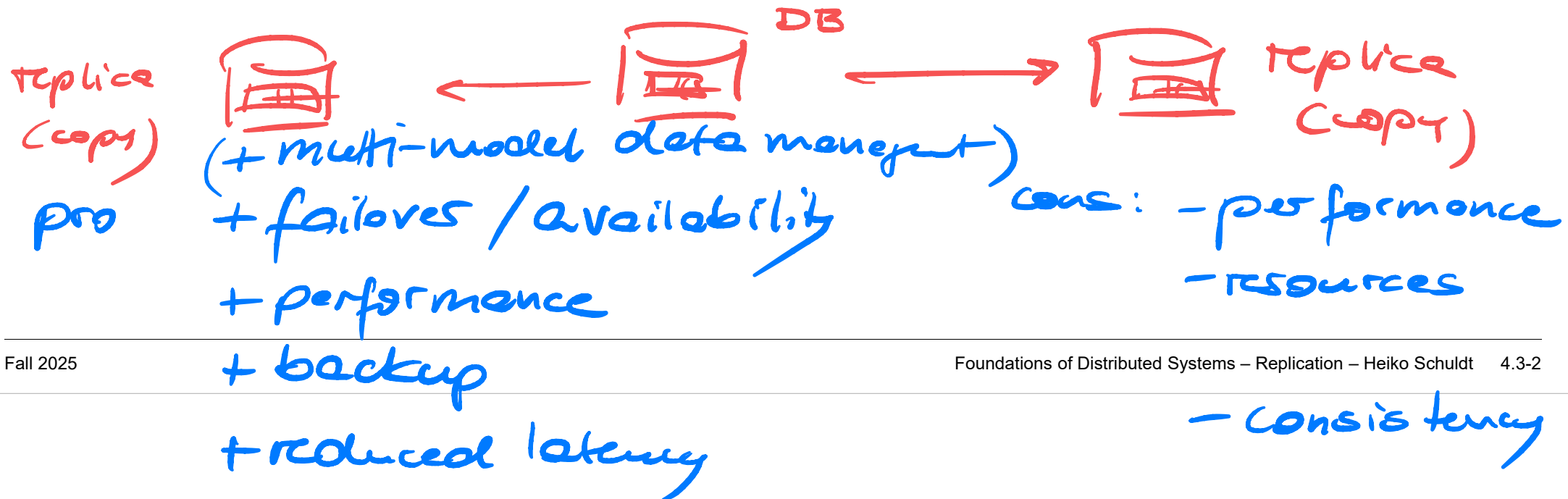
Fall Semester 2025

Prof. Dr. Heiko Schuldt



Overview of Chapter 4

4.1	Self-Organization (P2P)
4.2	Data Consistency (Distributed Transactions)
4.3	Data Replication
4.3.1	Replication Management
4.3.2	Data Replication: Where & When?



Objectives of Chapter 4.3 – Data Replication

- Replication (of data, but also of functionality) is an important concept in Computer Science to deal with failures – but is also creates new challenges to be addressed in terms of consistency.
- In this chapter, we focus on replication from a data perspective, and we will address the following questions:
 - What are the basic principles of distributed databases that replicate data across several nodes in a system?
 - Which approaches to replicate data and to manage replicas in a distributed systems exist?
 - How can updates to replicated data be coordinated?

You will ...

- distinguish the different approaches for data replication, especially in terms of handling updates
- be able to select the best suited approach for concrete requirements
- understand the consequences on transaction management and consistency guarantees when dealing with several versions of data.

Chapter 4.3.1 – Replication Management

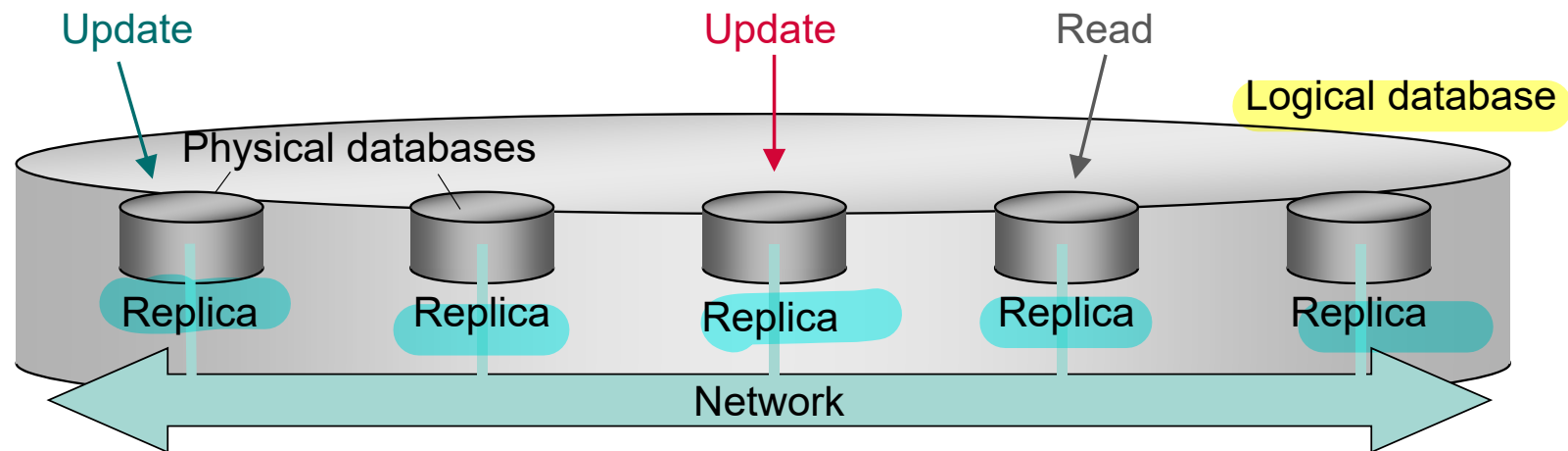
4.1	Self-Organization (P2P)
4.2	Data Consistency (Distributed Transactions)
4.3	Data Replication
4.3.1	Replication Management
4.3.2	Data Replication: Where & When?

Data Replication ...

- Data replication considers the storage of data objects at several (geographically) distributed storage systems / databases
- Data replication allows to ...
 - ... provide efficient access to large volumes of data by distributing the load across several replica sites
= increased performance
 - ... guarantee that data is available, despite of the failure of one (or several) systems / sites where data is located
= increased availability

... Data Replication

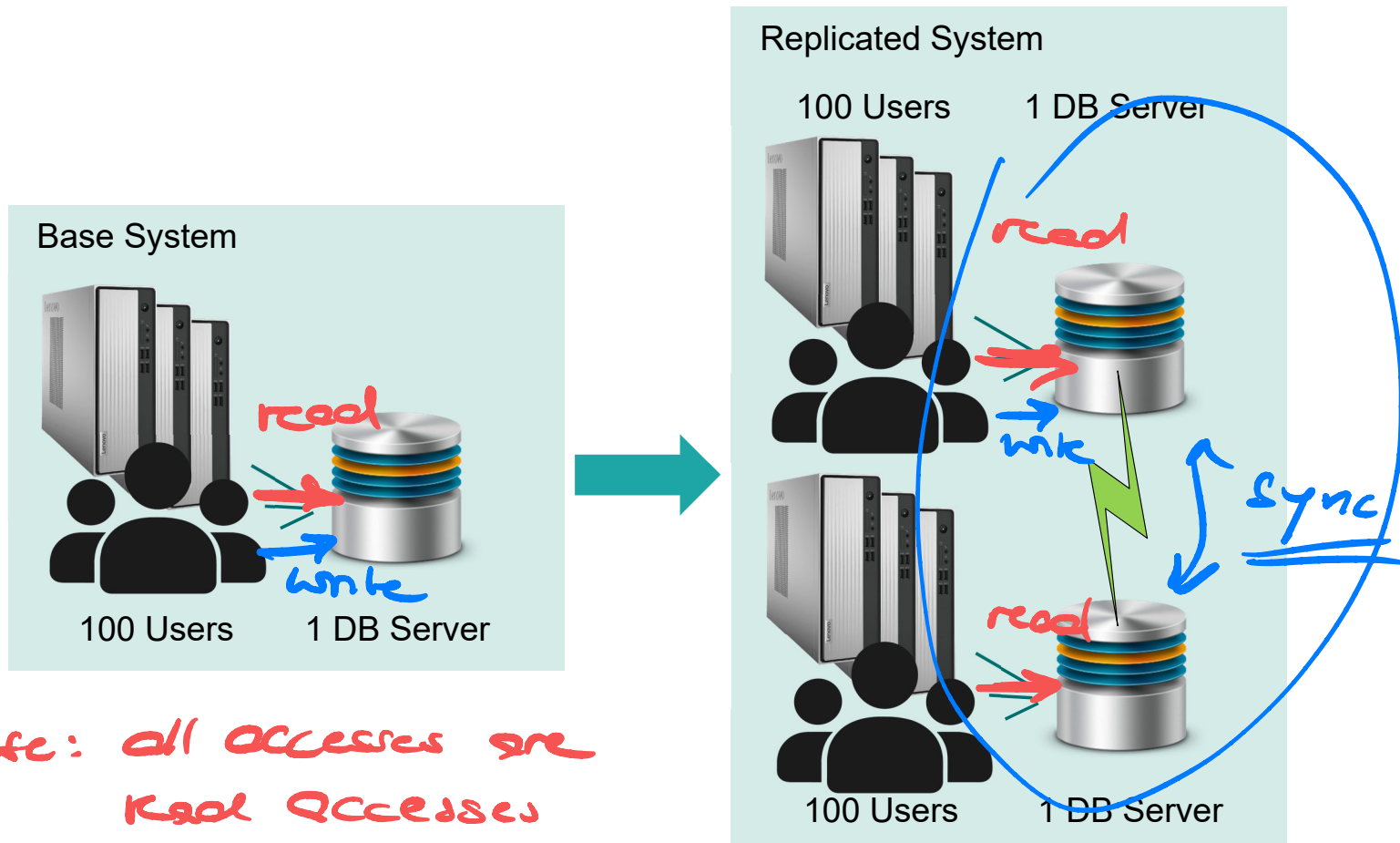
- In a replicated system, there is **one logical database** ...
- ... which is actually distributed across **several (n) replicated physical databases**.
- All replica nodes are connected via a network
- When accessing data, replication should be made transparent to the clients (i.e., they should be able to connect to a physical replica without having to be aware of the existence of other replicas)



Data Replication: Increased Scalability ...

- Scalability denotes the capability of a system to **cope with an increased number of users** (which also means an increased number of transactions, operations, etc.) if also the number of resources increases
- Hence, in the ideal case doubling the available resources would mean that the number of users that can be served doubles as well (which implies that there is no additional effort for managing / synchronizing the replicas).
- This ideal case is only possible in very special situations; nevertheless, replication usually increases the performance of the overall system.

... Data Replication: Increased Scalability



*ideal case: all accesses are
Read Accesses*

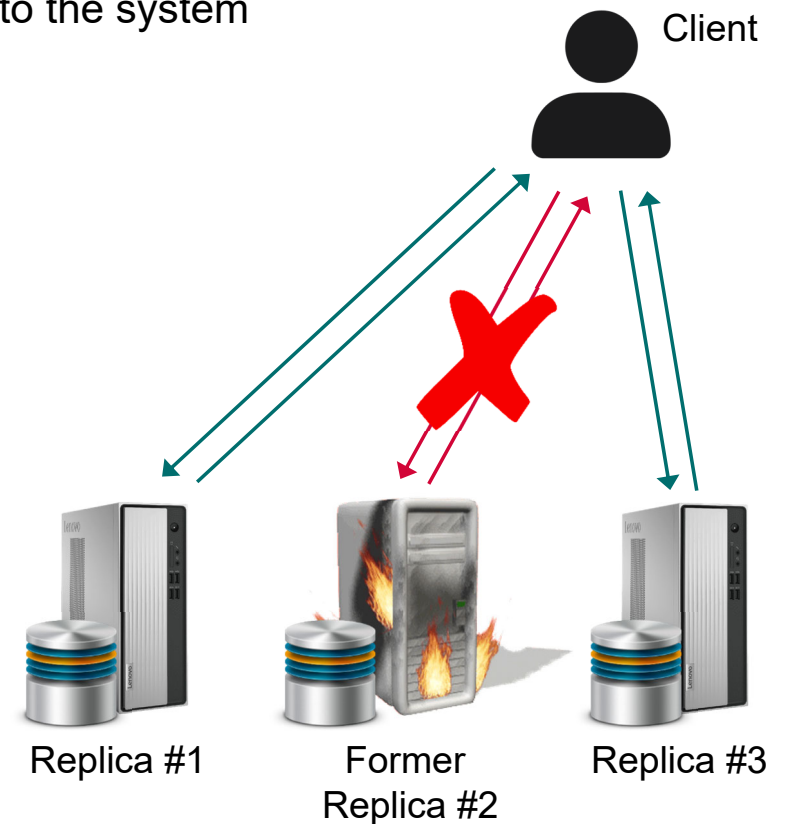
*but: no linear scalability in terms of write
requests*

Data Replication: Increased Availability

- The degree of data availability is increased by adding redundancy to the system (by means of exact copies of data sources)
 - A single component c fails with probability $P_{\text{single}}(c) = p$
 - For the probability P_{Rep} of a failure in a replicated system comprising n replicas, the following holds:

$$P_{\text{Rep}} = p^n$$

(based on the assumption that all replicas are independent of each other)



Data Replication and Updates

- Read operations in a replicated systems are not critical.
 - Since reads do not change the state of the overall system, no additional synchronization is necessary after a read.
- Updates, in contrast, require additional synchronization effort
 - Changes have to be propagated to all replica sites
 - Several approaches exist to deal with concurrent updates in a replicated system; these approaches differ in terms of the effort needed and the consistency guarantees that can be provided.

Chapter 4.3.2 – Data Replication: Where & When?

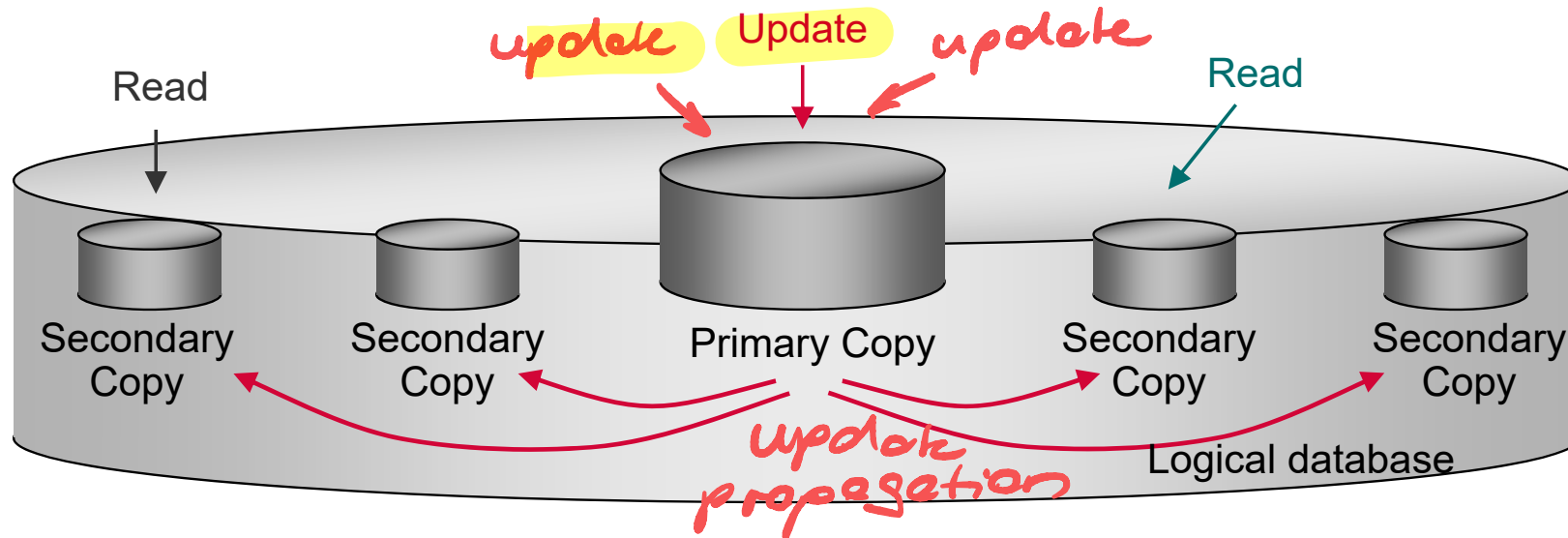
4.1	Self-Organization (P2P)
4.2	Data Consistency (Distributed Transactions)
4.3	Data Replication
4.3.1	Replication Management
4.3.2	Data Replication: Where & When?

Classification of Replication Protocols

- Replication protocols differ in terms of
 - **where** (i.e., at which replica node) updates can be executed
 - **when** changes are propagated to replicas
- Regarding the **location where updates can be executed**, a distinction is made between
 - **Primary Copy**: one dedicated replica node in the system hosts a „leading“ copy. All updates have to be applied at this copy and are then propagated to the **other replicas**
 - **Update Everywhere**: updates can be applied at each replica in the system
- Regarding **when updates are propagated**, the following approaches exist:
 - **Eager Replication**: the update operation and all subsequent update propagation is done **within the same transaction** — *one distributed transaction*
 - **Lazy Replication**: the original update operation and all updates that happen at the other replicas are logically **decoupled**, i.e., they are executed in different transactions — *two or more distributed transactions*

Primary Copy

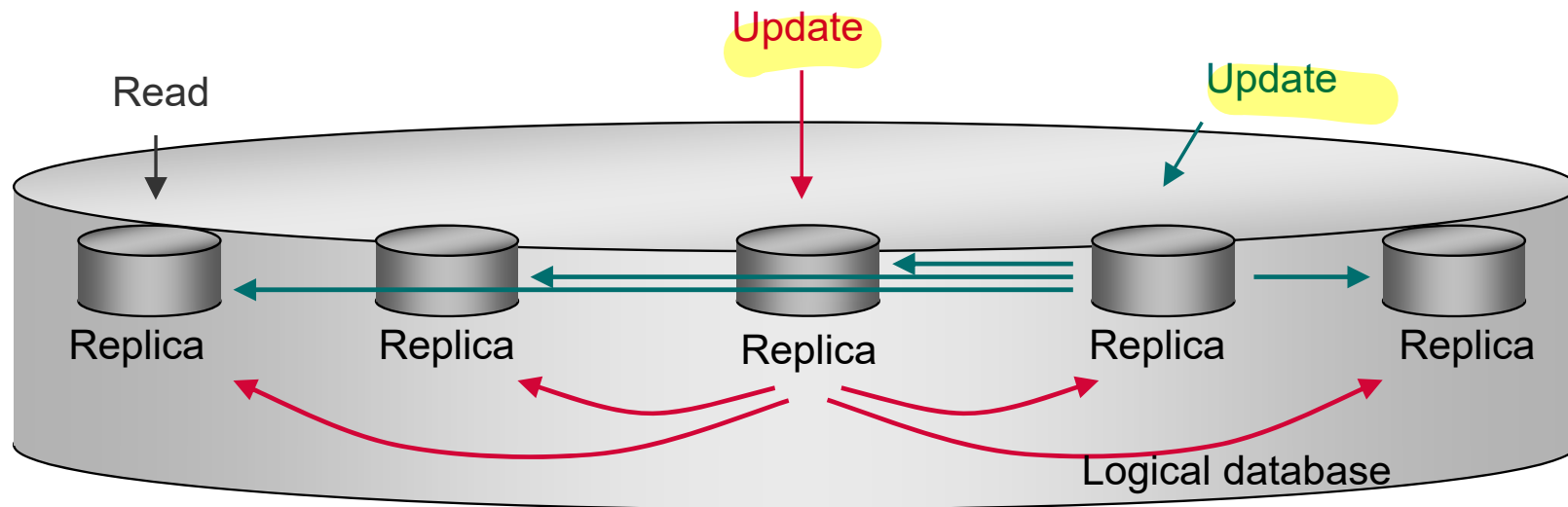
- With **primary copy replication**, (also called **“master copy”** approach), one replica (per data item) is the leading copy. All update operations are routed to this replica node. From there, the changes are distributed to all other replica nodes, which are called **secondary copies**. At these secondary copies, only read operations are allowed.



- The role of a primary copy can be taken over at the level of individual data items; i.e., a replica node A can act as a primary copy for data item x and as a read-only secondary copy of y, while at node B this is just the opposite.

Update Everywhere

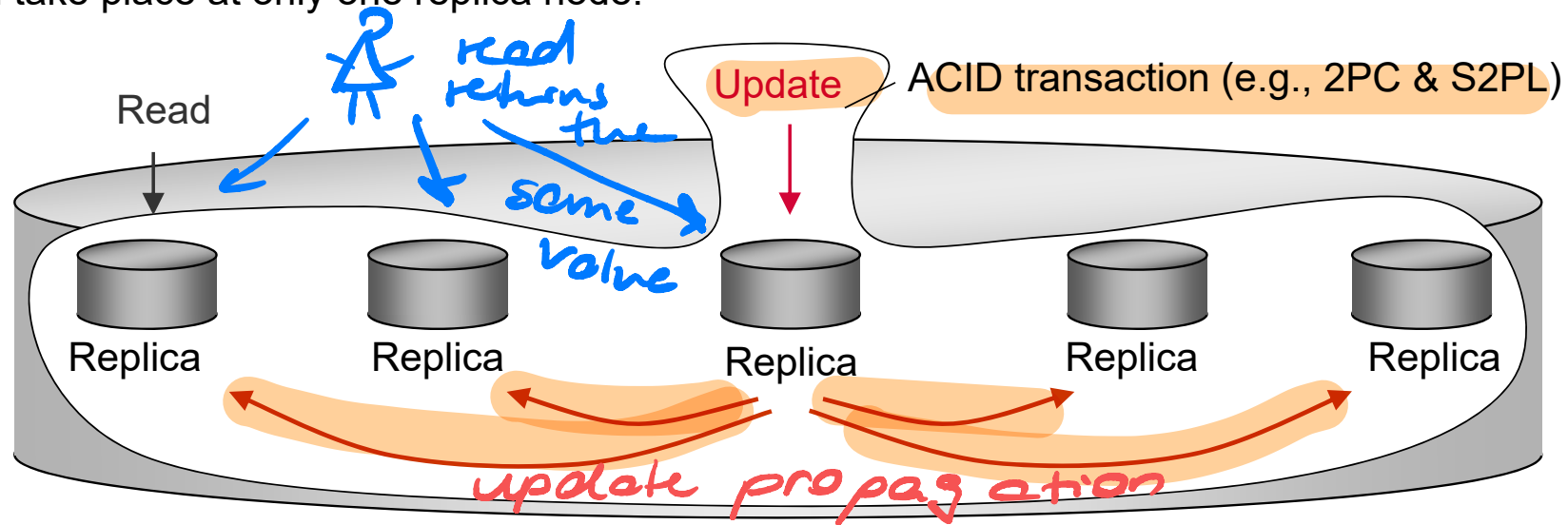
- In the **update everywhere approach**, all replica nodes are equal. Both updates and reads can take place on any replica node.



- With update everywhere, problems might occur when conflicting updates are done concurrently at different replica nodes. These conflicts have to be resolved (called reconciliation), based on dedicated rules (e.g., priorities of replica nodes, an order between nodes, when the update was submitted, etc.).

Eager Replication

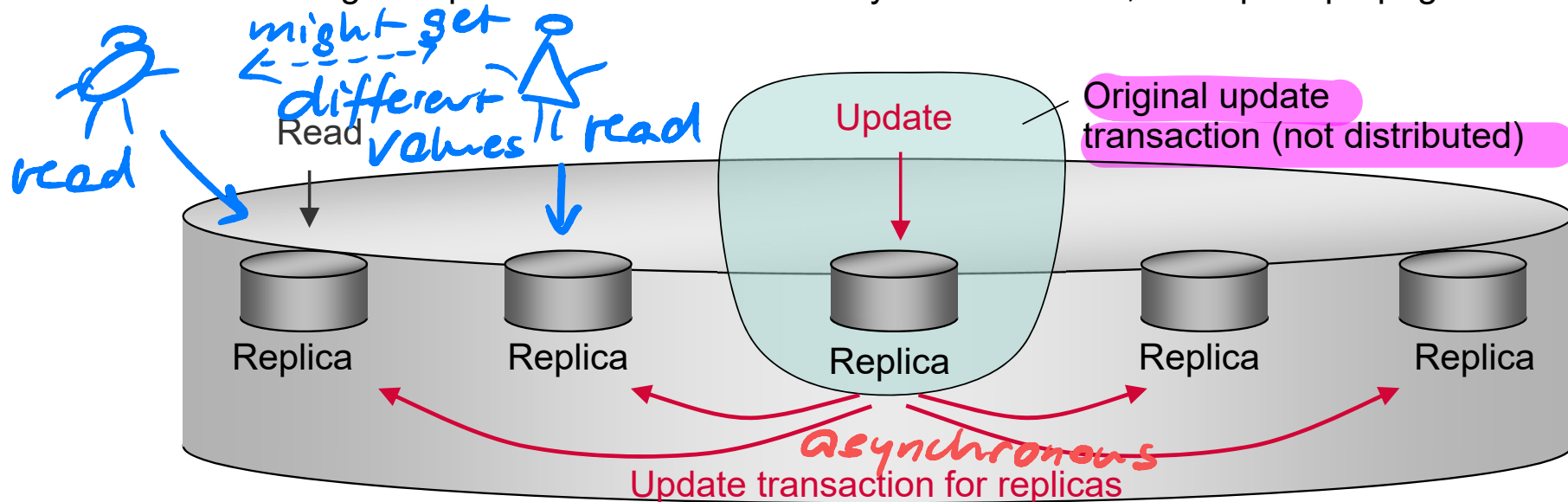
- With the **eager replication** approach (also called **synchronous replication**), ACID properties need to be jointly provided for both the actual update and the subsequent propagation of changes. This means that each update triggers a distributed atomic transaction (2PC and variants). While updates have to be sent to all replica nodes, reads can take place at only one replica node.



- Eager replication guarantees that all **replicas are consistent at any point in time**, i.e., they hold exactly the same data.

Lazy Replication

- With the **lazy replication** approach (also called **asynchronous replication**), the original update operation and the update propagation (presenting the update to the other replica nodes) take place in completely separated transactions. The original update transaction is usually not distributed, but replica propagation is.



- Since the original update is usually decoupled from the updates to the other replicas, it can commit earlier. However, inconsistencies between the original data and their replicas exist.

Classification of Replica Protocols

- Both distinctions are completely orthogonal.
- This means, there are in total four classes of data replication approaches

	+ avail. (read) Where can updates be executed?	
When are updates propagated?	+ consistency / correctness - availability (write) - performance	+ correctness - performance (reconciliation)
	+ performance - correctness Eager Primary Copy	+ performance (except for reconciliation) - correctness Lazy Update Everywhere

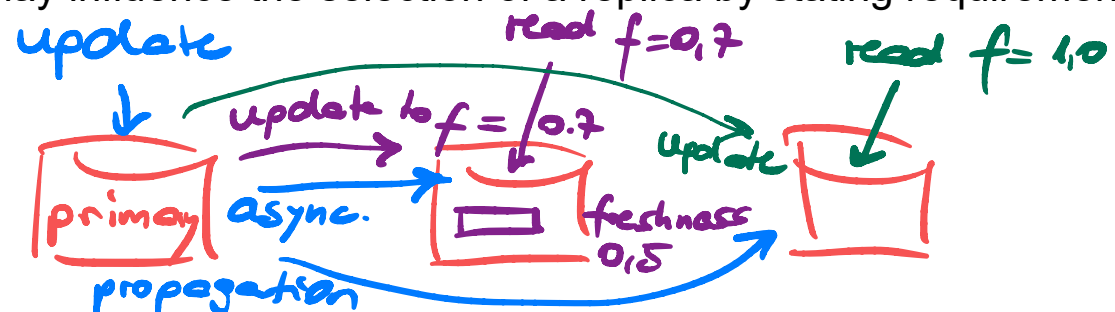
- correctness (only impacted until propagation)

- - - correctness

Discussion ...

Lazy vs. Eager:

- Eager replication is usually considered very „expensive“, due to the atomic commit at all replica nodes. In this case, consistency has to be realized by means of complex update operations.
- Lazy replication, in contrast, can be provided much easier (as updates always succeed even if not all replicas in the system are available). However, read operations have to be treated with special care since the data item that is read can already be outdated. Results of a read operation might return inconsistent data.
- Some novel approaches combine eager and lazy replication. The idea is to pretend to the outside that eager replication is applied while internally, only the cheaper lazy approach is used. This usually goes along with the introduction of a freshness parameter per data item that specifies whether a replica is up-to-date and, if it is not, how outdated it is. Readers then may influence the selection of a replica by stating requirements on the freshness level they demand.



... Discussion

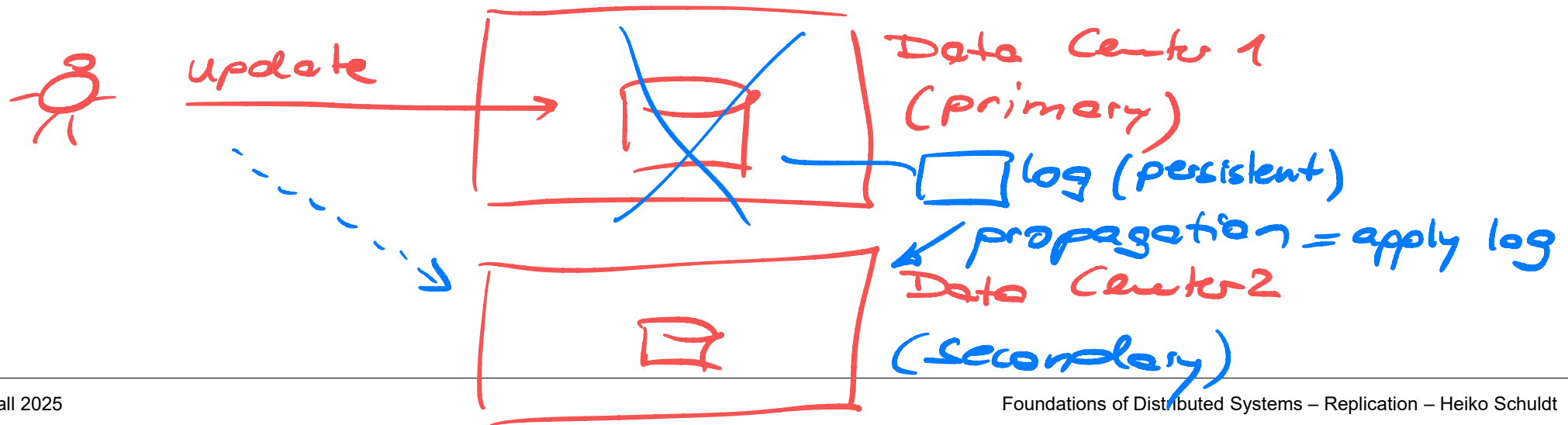
Primary vs. everywhere:

- The master copy approach does not provide much flexibility.  master copy approach Single point of failure
Fault tolerance is only provided for read operations: if the master node fails, then reads can be re-routed to any other replica node; however, in this case updates cannot be processed.
- Update everywhere provides a very high degree of fault tolerance as both read and update operations can be executed as long as at least one physical replica is available. In general, reconciliation is quite complex, and can usually only be provided with strict rules (the combination of update everywhere and lazy replication is usually considered dangerous!).
- For reconciliation, timestamps can be used (to approve older updates and discard younger ones in case of a conflict). But this just delegates the problem to clock synchronization ...

Examples of Replication Protocols ...

Cold standby: Primary Copy – Lazy Replication

- This approach is used in client/server architectures for a server-side backup (so-called “cold standby”)
- Updates are executed on the primary copy. These updates are **persistently logged outside of this server** (e.g., in a persistent queue), but not immediately applied at the other replica nodes.
- In case the server fails, **first all updates from the queue have to be re-played at a secondary replica**. As soon as this (previously) secondary replica is up-to-date, it can replace the failed server and will become the new primary copy.

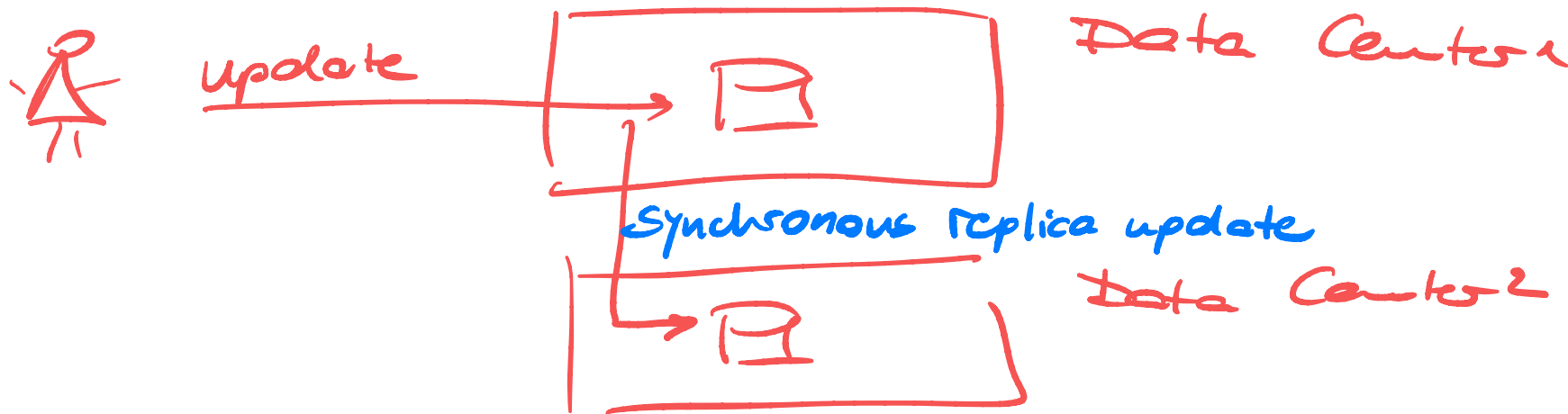


... Examples of Replication Protocols ...

Hot standby: Primary Copy – Eager Replication

- This approach is used in client/server architectures for server-side backup, especially in systems where all transactions have to be immediately re-routed to another system in case of failures (so-called “hot standby”)
- Updates are executed at the primary node and are then propagated within the same transaction to all replicas. In order to keep availability high, it is recommended to keep the number of replicas low.
- If the server fails, requests can immediately be re-routed without additional coordination effort.

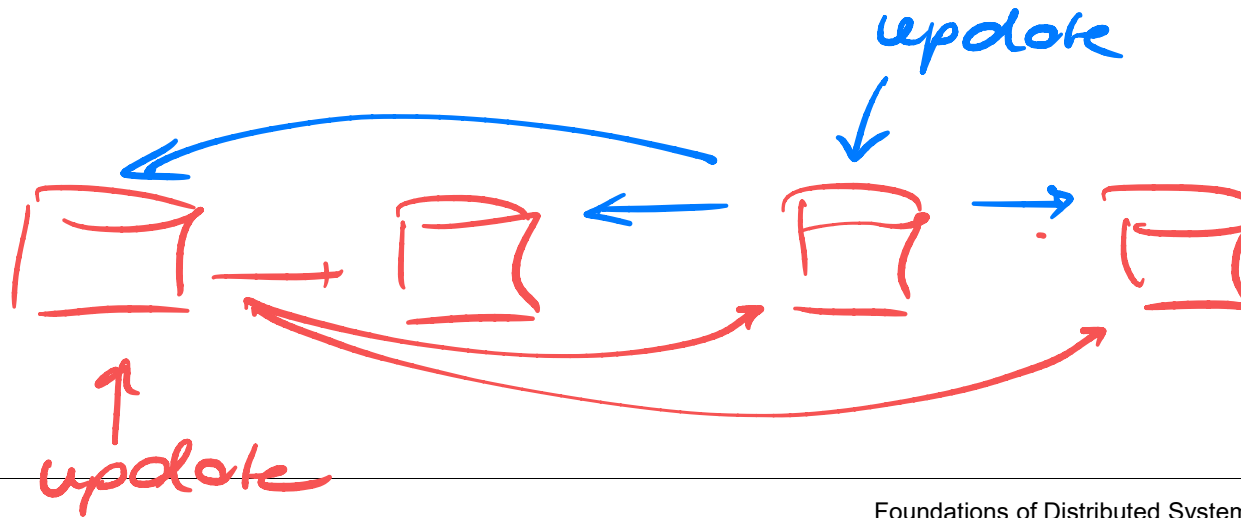
This approach allows an organization to operate their business without interrupt.



... Examples of Replication Protocols ...

Read One Write All (ROWA): Update Everywhere – Eager Replication

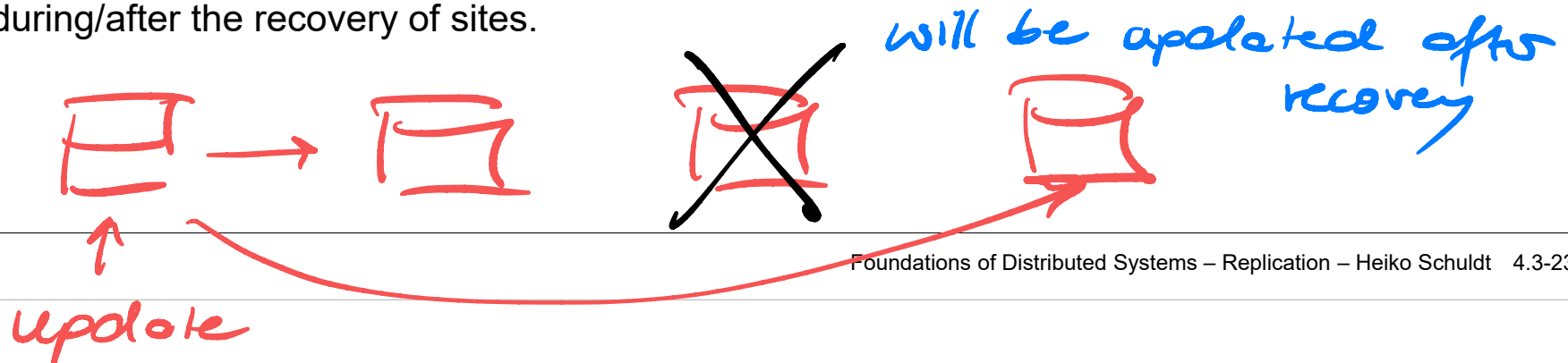
- This is the traditional model when considering replication in a distributed system with equal data sources.
- Updates can potentially be executed anywhere – but these updates only succeed if they are globally accepted in the system.
- This approach is well suited in systems where the number of reads significantly exceeds the number of writes. If, in a system that uses locks, too many update requests come to the system at the same time, the probability of running into a deadlock cannot be neglected; performance would be seriously impacted.



... Examples of Replication Protocols

Read One Write All Available (ROWAA): Update (mostly) Everywhere – Combination of Eager and Lazy Replication, with integrated fault tolerance

- First, an eager update is performed – if possible on all sites. If some sites are not available, only the ones that are on-line (available) are considered for the eager replication part of ROWAA
- Second, the sites that were down at commit time of a transaction are updated in a lazy fashion as soon as they have recovered
- Updates can potentially be executed anywhere, and these updates succeed if they are accepted by all available replicas in the system.
- This approach is well suited in cases where eager replication is in general beneficial over other approaches and in which it cannot be guaranteed that all replicas are available at each point in time. In the traditional eager approach, an update transaction would fail if at least one replica site is not available. This approach is much more tolerant so site failures. However, the price that has to be paid for that is increased effort during/after the recovery of sites.



Further Approaches

- **Pull-based replication**: each replica is responsible for checking whether there are new updates, and then to locally apply them
- **Push-based replication**: it is the task of the server to propagate (to „push“) replicas.
 - **Unicast**: changes are sent directly to one recipient. If there are several replicas, each have to be addressed separately.
 - **Multicast**: changes can be jointly sent to all recipients (replicas)
- **Epidemic protocols**: can be applied to any type of large system where not all replicas are known. Each node „knows“ some other nodes in its neighborhood and forwards updates to these neighbors for execution (this is called infection). The infected replicas, in turn, may keep forwarding updates as long as all nodes have received this information. Hence, changes are gradually propagated in the entire system.
- **Quorum protocols**: several (not necessarily all) replicas need to be considered both for write and also for read operations.

→ P2P
Systems

Quorum Protocols ...

Let:

- N be the total number of nodes (= replica sites) in a system,
- R be the number of sites from which a transactions reads,
- W be the number of sites a transaction writes to.

Then, existing replication protocols can be characterized as follows:

- ROWA (eager, update everywhere): $R = 1, W = N$
- ROWAA: $R = 1, W = N_{\text{avail}}$, with $N_{\text{avail}} > N/2$. $\equiv$ *Quorum*

But: The non-available sites ($N - N_{\text{avail}}$) cannot be considered for further reads until the changes have been propagated.

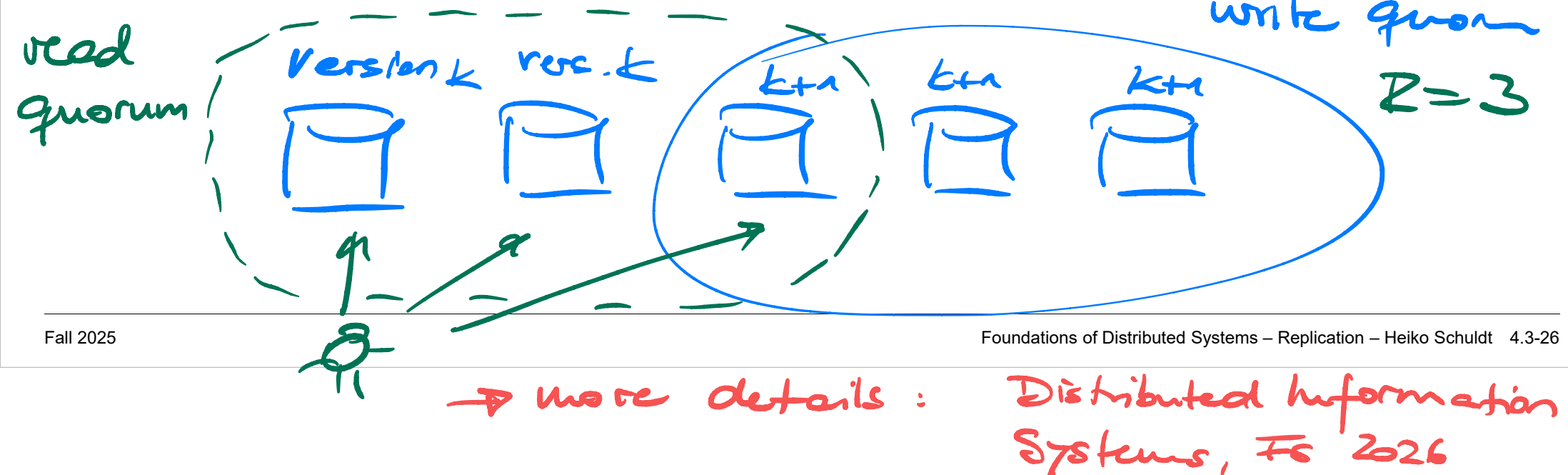
... Quorum Protocols

- Quorum protocols further generalize the handling of reads and updates
- In general, the less sites are used for eager updates, the more sites to be considered when reading an object
- Rules for correct replication in a general quorum:
 - $W > N/2$ (Write Quorum): eagerly update at least half of the sites
There must not be another concurrent write quorum
 - $R > N - W$ (Read Quorum): do not miss the latest data ($R + W > N$)
- Writes can be performed more easily, but reading is more expensive.

$$N=5$$
$$W=3$$

write quorum

$$R=3$$



Recap: Questions

- When can a distributed system benefit from replicated data?
- What are the special challenges when replicating data, in which situations?
- What are the advantages and disadvantages of different replications protocols (regarding the question where and when to propagate updates)?
- How are quorum protocols defined? How do quorum protocols relate to eager and lazy replication?

Further Reading

- [Kem 18] B. Kemme: *One-Copy-Serializability*. In: Encyclopedia of Database Systems (2nd ed.), Springer, 2018.
- [WV 02] G. Weikum, G. Vossen: *Transactional Information Systems*, Morgan Kaufmann, 2001.